

Complexity Analysis - USEI06

It'll be considered:

- **n** -> total number of stations (< 64.000);
- **m** -> total number of nodes in the tree;
- **s** -> average of stations per node (n/m);
- **k** -> number of results;
- **l** -> number of different lat/lon;
- **t** -> number of different timezone groups (= 4);
- **c** -> number of different countries (= 44).

Tree methods:

```
int height(Node<E> node) {  
    return (node == null) ? -1 : node.getHeight();  
}
```

- Optimized version of height(node) method.
- Getter call: O(1).
- Total: O(1).
- Returns the height of a node using a stored value. Runs in constant time because it only checks null and calls a getter.

```
Node<E> balanceNode(Node<E> node) {  
    int bf = balanceFactor(node);  
  
    if (bf > 1) {  
        if (balanceFactor(node.getLeft()) < 0) {  
            node.setLeft(leftRotation(node.getLeft()));  
        }  
        node = rightRotation(node);  
    } else if (bf < -1) {  
        if (balanceFactor(node.getRight()) > 0) {  
            node.setRight(rightRotation(node.getRight()));  
        }  
        node = leftRotation(node);  
    }  
}
```

```

    }

    return node;
}

```

- balanceFactor(node) call: $O(1)$.
- Rotation if necessary: $O(1)$.
- Total: $O(1)$.
- Balances an AVL node by checking its balance factor and performing rotations if needed. Each rotation and check is constant time, so overall complexity is $O(1)$.

```

void updateHeight(Node<E> node) {
    if (node != null) {
        int leftHeight = (node.getLeft() == null) ? -1 :
node.getLeft().getHeight();
        int rightHeight = (node.getRight() == null) ? -1 :
node.getRight().getHeight();
        node.setHeight(1 + Math.max(leftHeight, rightHeight));
    }
}

```

- Getters: $O(1)$.
- Setter call: $O(1)$.
- Total: $O(1)$.
- Updates the height of a node based on its children. Uses simple getter/setter operations, so it runs in $O(1)$.

```

Node<E> insert(E element, Node<E> node) {
    if (node == null) {
        return new Node<>(element, null, null);
    }
    if (element.compareTo(node.getElement()) == 0) {
        return node;
    }
    else if (element.compareTo(node.getElement()) < 0)
        node.setLeft(insert(element, node.getLeft()));
}

```

```

else
    node.setRight(insert(element, node.getRight()));

return node;
}

```

- Search for the correct position to insert the new element:
O(log m).
- balanceNode(node) calls: O(1).
- updateHeight(node) call: O(1).
- Total: O(log m).
- Inserts an element into an AVL tree recursively. The search for the correct position takes O(log m) in a balanced tree, followed by constant-time updates.

```

Node<E> remove(E element, Node<E> node) {

    if (node == null) {
        return null;
    }
    if (element.compareTo(node.getElement())==0) {
        if (node.getLeft() == null && node.getRight() == null) {
            return null;
        }
        if (node.getLeft() == null) {
            return node.getRight();
        }
        if (node.getRight() == null) {
            return node.getLeft();
        }
        E min = smallestElement(node.getRight());
        node.setElement(min);
        node.setRight(remove(min, node.getRight()));
    }
    else if (element.compareTo(node.getElement()) < 0)
        node.setLeft( remove(element, node.getLeft()) );
    else
        node.setRight( remove(element, node.getRight()) );
}

```

```

    return node;
}

```

- Search for the element to remove: $O(\log m)$.
 - Remotion and adjustment: $O(\log m)$
 - `balanceNode(node)` calls: $O(1)$.
 - `updateHeight(node)` call: $O(1)$.
 - Total: $O(\log m)$.
 - Removes an element from an AVL tree recursively. Searching and restructuring both take $O(\log m)$ in a balanced tree.
-

```

E search(Node<E> node, E element) {
    if (node == null) return null;

    int cmp = element.compareTo(node.getElement());

    if (cmp == 0) {
        return node.getElement();
    } else {
        if (cmp < 0) {
            return search(node.getLeft(), element);
        } else {
            return search(node.getRight(), element);
        }
    }
}

```

- Searching for the element: $O(\log m)$.
 - Total: $O(\log m)$.
 - Finds an element in the tree using binary search logic. Complexity is $O(\log m)$ for balanced trees.
-

```

Node<E> find(Node<E> node, E element) {
    if (node == null)
        return null;

    if (element.compareTo(node.getElement()) == 0) {

```

```

        return node;
    }
    else if (element.compareTo(node.getElement()) < 0)
        return find(node.getLeft(), element);
    else
        return find(node.getRight(), element);
}

```

- Searching for the node: same logic as the search(node, element) method -> $O(\log m)$.
 - Total: $O(\log m)$.
 - Similar to search(), but returns the node instead of the element. Complexity remains $O(\log m)$.
-

Comparators methods:

```

void addStation(RailwayStation station) {
    stations.add(station);
    sorted = false;
}

```

- For each station: $O(1)$.
 - Total (for all stations): $O(s) \approx O(n)$.
 - Adds a station to a list. Each addition is $O(1)$.
-

```

void ensureSorted() {
    if (!sorted) {

stations.sort(Comparator.comparing(RailwayStation::getName));
        sorted = true;
    }
}

```

- Firstly, for sorting: $O(s \log s) \approx O(n \log n)$.
- Subsequent calls after the first one: $O(1)$.

- Sorts the station list if not already sorted. First call costs $O(n \log n)$; subsequent calls are $O(1)$.

```
(CountryIndex)
void buildTree() {
    if (isBuilt) return;

    stations = new AVLTree<>();

    tempStations.sort(Comparator.comparing(RailwayStation::getName))
;

    for (RailwayStation station : tempStations) {
        stations.insert(new RailwayStationByName(station));
    }

    tempStations = null;
    isBuilt = true;
}
```

- Sorting of tempStations: $O(s \log s) \approx O(n \log n)$.
- Loop (for s stations, and taking into account that the insertion process has $O(\log s)$ complexity): $O(s \log s)$.
- Total: $O(s \log s)$.
- Sorts stations and inserts them into an AVL tree.

```
void inOrderTraversal(AVLTree.Node<RailwayStationByName> node,
List<RailwayStation> acc) {
    if (node == null) return;
    inOrderTraversal(node.getLeft(), acc);
    acc.add(node.getElement().station());
    inOrderTraversal(node.getRight(), acc);
}
```

- Runs through the left subtree: $O(s/2)$.
- Runs through the right subtree: $O(s/2)$.
- Total: $O(s/2 + s/2) = O(s)$.

- Traverses the tree in order and collects elements.
Complexity is $O(n)$ because it visits all nodes.
-

```
List<RailwayStation> getStationsInOrder() {
    ensureBuilt();
    List<RailwayStation> ordered = new ArrayList<>();
    inOrderTraversal(stations.root(), ordered);
    return ordered;
}
```

- inOrderTraversal call: $O(s)$.
 - Total: $O(s)$.
 - Calls in-order traversal and returns the list. Complexity is $O(n)$.
-

```
void addStation(RailwayStation station) {
    ensureBuilt();
    Country country = station.getCountry();
    CountryIndex countryIndex = new CountryIndex(country);
    AVLTree.Node<CountryIndex> foundNode =
countries.find(countries.root(), countryIndex);

    CountryIndex actual;
    if (foundNode == null) {
        countries.insert(countryIndex);
        actual = countryIndex;
    } else {
        actual = foundNode.getElement();
    }

    actual.addStation(station);
}
```

- ensureBuilt(): First call $\rightarrow O()$ | Most of calls: $O(1)$.
- find call: $O(\log c) = O(\log m) \approx O(1)$.
- insert call: $O(\log c) = O(\log m) \approx O(1)$.
- addStation call: $O(\log s) \approx O(\log n)$.

- Total: $O(\log s) \approx O(\log n)$.
- Adds a station to the correct country index. Involves searching and inserting in AVL trees, costing $O(\log n)$ overall.

```

(TimeZoneIndex)
void buildTree() {
    if (isBuilt) return;

    countries = new AVLTree<>();

    for (CountryIndex countryIndex : tempCountries.values()) {
        countryIndex.getStationsInOrder();

        countries.insert(countryIndex);
    }

    tempCountries = null;
    isBuilt = true;
}

```

- getStationsInOrder() call: $O(n)$ -> already considering the loop.
 - insert call: $O(c \log c)$ -> already considering the loop.
 - Total: $O(n)$.
 - Builds a tree of countries and their stations.
-

Repository methods:

```

void addStation(RailwayStation station) {
    double lat = station.getLatitude();
    RailwayStationLatitude latWrapper = new
RailwayStationLatitude(lat);
    var latNode = latitudeTree.find(latitudeTree.root(),
latWrapper);

    if (latNode == null) {
        latWrapper.addStation(station);
    }
}

```

```

        latitudeTree.insert(latWrapper);
    } else {
        latNode.getElement().addStation(station);
    }

    double lon = station.getLongitude();
    RailwayStationLongitude lonWrapper = new
RailwayStationLongitude(lon);
    var lonNode = longitudeTree.find(longitudeTree.root(),
lonWrapper);

    if (lonNode == null) {
        lonWrapper.addStation(station);
        longitudeTree.insert(lonWrapper);
    } else {
        lonNode.getElement().addStation(station);
    }

    TimeZoneIndex tzIndex = new
TimeZoneIndex(station.getTimeZoneGroup());
    var tzNode =
timeZoneCountryTree.find(timeZoneCountryTree.root(), tzIndex);

    TimeZoneIndex actual;
    if (tzNode == null) {
        timeZoneCountryTree.insert(tzIndex);
        actual = tzIndex;
    } else {
        actual = tzNode.getElement();
    }

    actual.addStation(station);
}

```

- "latitudeTree.find(latitudeTree.root(), latWrapper)": $O(\log 1)$.
- "longitudeTree.find(longitudeTree.root(), lonWrapper)": $O(\log 1)$.
- "timeZoneCountryTree.find(timeZoneCountryTree.root(), tzIndex)": $O(\log t) \approx O(1)$.
- "latitudeTree.insert(latWrapper)": $O(\log 1)$.

- `"longitudeTree.insert(lonWrapper)":  $O(\log l)$ .`
 - `"timeZoneCountryTree.insert(tzIndex)":  $O(\log t) \approx O(1)$ .`
 - `"actual.addStation(station)":  $O(\log n)$ .`
 - Total: $O(\log l + \log n)$.
 - Updates multiple trees (latitude, longitude, time zone).
Each search and insertion is logarithmic, so total is $O(\log l + \log n)$.
-

```
void latitudeRangeQuery(BST.Node<RailwayStationLatitude> node,
double min, double max, List<RailwayStationLatitude> result) {
    if (node == null)
        return;

    double lat = node.getElement().getLatitude();

    if (lat >= min && lat <= max) {
        result.add(node.getElement());
    }

    if (lat >= min) {
        latitudeRangeQuery(node.getLeft(), min, max, result);
    }

    if (lat <= max) {
        latitudeRangeQuery(node.getRight(), min, max, result);
    }
}
```

- For the nodes with latitude inside the range: $O(k)$.
 - For the nodes with latitude outside the range: $O(\log l)$.
 - Total: $O(k + \log l)$.
 - Performs range queries on AVLTrees based on a range of latitude. Complexity is $O(k + \log l)$, where k is the number of matching nodes.
-

```
void longitudeRangeQuery(BST.Node<RailwayStationLongitude> node,
double min, double max, List<RailwayStationLongitude> result) {
```

```

if (node == null)
    return;

double lon = node.getElement().getLongitude();

if (lon >= min && lon <= max) {
    result.add(node.getElement());
}

if (lon >= min) {
    longitudeRangeQuery(node.getLeft(), min, max, result);
}

if (lon <= max) {
    longitudeRangeQuery(node.getRight(), min, max, result);
}
}

```

- For the nodes with longitude inside the range: $O(k)$.
- For the nodes with longitude outside the range: $O(\log l)$.
- Total: $O(k + \log l)$.
- Same logic as the previous method, performs range queries on AVL Trees but now based on a range of longitude.

```

List<Country> filterCountries(TimeZoneGroup tzGroup) {
    TimeZoneIndex key = new TimeZoneIndex(tzGroup);
    TimeZoneIndex ti = timeZoneCountryTree.search(key);

    if (ti == null)
        return Collections.emptyList();

    Set<Country> countries = new TreeSet<>();
    for (CountryIndex ci : ti.getCountries().inOrder()) {
        countries.add(ci.getCountry());
    }

    return new ArrayList<>(countries);
}

```

- search(key) call: $O(\log t) \approx O(1)$.
 - inOrder() call: $O(c)$ ('c' means the quantity of different countries in a time zone group).
 - Insertion in a TreeSet: $O(\log c)$.
 - Total: $O(c) * O(\log c) = O(c \log c)$.
 - Retrieves countries for a time zone group and sorts them.
-

```
List<RailwayStation> searchByTimeZoneAndCountry(TimeZoneGroup
tzGroup, Country country) {
    List<RailwayStation> result = new ArrayList<>();
    AVLTree.Node<TimeZoneIndex> tzNode =
timeZoneCountryTree.find(timeZoneCountryTree.root(), new
TimeZoneIndex(tzGroup));

    if (tzNode != null) {
        TimeZoneIndex tzIndex = tzNode.getElement();
        AVLTree.Node<CountryIndex> countryNode =
tzIndex.getCountries().find(tzIndex.getCountries().root(), new
CountryIndex(country));
        if (countryNode != null) {

result.addAll(countryNode.getElement().getStationsInOrder());
        }
    }

    return result;
}
```

- find(root(), timeZoneIndex) call: $O(\log t) \approx O(1)$.
 - find(root(), countryIndex) call: $O(\log c) \approx O(1)$.
 - addition of stations in result list: $O(s)$.
 - Total: $O(s)$.
 - Finds stations by time zone and country.
-

```
void windowQuery(AVLTree.Node<TimeZoneIndex> node, TimeZoneGroup
lower, TimeZoneGroup upper, List<RailwayStation> result) {
```

```

    if (node == null || node.getElement() == null ||
node.getElement().getTimeZoneGroup() == null) return;

    TimeZoneGroup tz = node.getElement().getTimeZoneGroup();

    if (tz.compareTo(lower) > 0) {
        windowQuery(node.getLeft(), lower, upper, result);
    }

    if (tz.compareTo(lower) >= 0 && tz.compareTo(upper) <= 0) {
        Iterable<CountryIndex> countries =
node.getElement().getCountries().inOrder();

        for (CountryIndex ci : countries) {
            result.addAll(ci.getStationsInOrder());
        }
    }

    if (tz.compareTo(upper) < 0) {
        windowQuery(node.getRight(), lower, upper, result);
    }
}

```

- Loop:
 - inOrder() call: $O(c_{tz})$ ('c_tz' means all the countries in a specific time zone group).
 - getStationsInOrder() call: $O(s)$.
 - Total: $O(c) * O(s) = O(c_{tz} * s)$ per time zone group.
 - Navigating through the different time zone groups: $O(k + \log t) \approx O(k)$, being 'k' the number of time zone groups inside the range (maximum value of 'k' is 4).
 - Total: $O(k * c_{tz} * s)$, being $k * c_{tz} * s$ the total number of stations returned for the range query.
 - Retrieves stations based on a window query of time zone groups.
-